

## 第 7 卷 调试、对拍与训练

考试速查：主查最后救命检查：边界样例、随机对拍、RE/WA/TLE 排查、提交前检查清单和赛后训练方法。

## Contents

|                                           |           |
|-------------------------------------------|-----------|
| <b>第 7 卷 调试、对拍与训练</b>                     | <b>2</b>  |
| 考试速查定位                                    | 2         |
| <b>TRAIN-00 第 7 卷：通用 WA / RE / TLE 自检</b> | <b>2</b>  |
| 1. 90 秒错误定位                               | 2         |
| 2. 通用 WA 检查                               | 3         |
| 3. 通用 RE 检查                               | 3         |
| 4. 通用 TLE 检查                              | 3         |
| 5. 提交前 7 问                                | 3         |
| 6. 失败后的回退路线                               | 4         |
| <b>TRAIN-01 第 7 卷：极端样例与反例库</b>            | <b>4</b>  |
| 1. 极端样例构造表                                | 4         |
| 2. 图论反例                                   | 5         |
| 3. DP 反例                                  | 5         |
| 4. 数据结构反例                                 | 5         |
| 5. 字符串 / 数学反例                             | 5         |
| 6. 最小反例构造口诀                               | 6         |
| <b>TRAIN-02 第 7 卷：题面摘要路由训练</b>            | <b>6</b>  |
| 训练使用法                                     | 7         |
| 01 静态区间和                                  | 7         |
| 02 单点修改区间和                                | 7         |
| 03 区间加最后输出                                | 7         |
| 04 大坐标排名统计                                | 7         |
| 05 逆序对                                    | 7         |
| 06 无权网格最少步数                               | 8         |
| 07 非负权单源最短路                               | 8         |
| 08 有负边小图最短路                               | 8         |
| 09 最小生成树                                  | 8         |
| 10 DAG 任务依赖                               | 8         |
| 11 树上路径距离                                 | 9         |
| 12 0/1 背包                                 | 9         |
| 13 完全背包                                   | 9         |
| 14 两个字符串匹配                                | 9         |
| 15 区间合并                                   | 9         |
| 16 访问所有关键点                                | 10        |
| 17 单模式字符串查找                               | 10        |
| 18 组合数取模                                  | 10        |
| <b>v0.2 本卷例题训练区</b>                       | <b>10</b> |
| V07-EX01 多组图连通块清空                         | 10        |
| V07-EX02 前缀和暴力核验                          | 12        |
| V07-EX03 背包循环顺序反例                         | 13        |
| V07-EX04 最短路双算法核验                         | 14        |
| V07-EX05 KMP 与暴力重叠核验                      | 16        |
| V07-EX06 空栈保护括号匹配                         | 17        |
| V07-EX07 乘法溢出上限判断                         | 18        |
| V07-EX08 二分答案边界核验                         | 19        |
| V07-EX09 RMQ 暴力对拍                         | 20        |

V07-CEX01 快慢算法核验逆序对 . . . . . 22
V07-CEX02 输入范围守卫 . . . . . 22
V07-CEX03 二分边界可视化 . . . . . 23
V07-CEX04 乘法溢出探针 . . . . . 23
V07-CEX05 空容器访问保护 . . . . . 23

第 7 卷 调试、对拍与训练

考试速查定位

第 7 卷 调试、对拍与训练

- 这是第几卷： 07。
• 主要查什么：主查最后救命检查：边界样例、随机对拍、RE/WA/TLE 排查、提交前检查清单和赛后训练方法。
• 什么时候翻：样例过但 WA/RE/TLE、需要自造极限数据或赛后训练时翻。
• 题面关键词：边界；对拍；随机数据；提交前检查；常见错误。

自动由 TRAIN 模块重建。定位是考场自查、反例库和小规模验证。

TRAIN-00 第 7 卷：通用 WA / RE / TLE 自检

模块编号：TRAIN-00

模块名称：通用 WA / RE / TLE 自检

标签：调试、提交前检查、WA、RE、TLE、部分分

一句话用途：提交前用 3 分钟扫掉最常见错误，失败后能快速判断是模型错、边界错还是复杂度错。

题面触发词：样例过了但不放心、隐藏点 WA、运行错误、超时、多组数据、边界、无解、极大数据。

什么时候用：每次提交前；从暴力升级到优化版后；出现 WA/RE/TLE 后不知道从哪查时。

不要什么时候用：不要用本清单代替重新读题；如果题面模型已经选错，改小 bug 救不了正解。

复杂度：检查清单本身无复杂度。

数据范围参考：所有题目。

依赖的标准容器：Array、Graph、Query、State、Compressor，以及各卷模块的统一接口。

输入如何整理：先圈 n/m/q/T/值域/是否多测/是否有修改/边权符号/是否取模/无解输出。

接口：提交前检查顺序。

输出能力：定位 WA、RE、TLE 高风险点，并给出回退到部分分版本的路线。

下游可接：TRAIN-01 反例库、TRAIN-02 路由训练、05\_review 质检。

可拼接模块：ROUTE-00、ROUTE-01、CPP-009、BRUTE-15、DP 常见坑、DS/GRAPH/MATH/STR 模块。

模板代码：无。本模块是检查卡。

调用示例：样例 AC 后，按 “WA -> RE -> TLE -> 最小反例 -> 提交版本” 顺序扫一遍。

常见坑：只测样例；只测随机不测边界；升级版改坏输入层；忘记多测清空；答案类型用 int。

暴力/部分分替代：如果优化版不稳，保留暴力或记忆化版本，先提交能过小数据的版本。

升级方向：把最小反例加入本地自测；用暴力和优化版对拍小数据；把可疑高级结构退回稳妥结构。

最小测试样例：n=1、空/无解、最大值、全相等、重复边/重复值、多组数据两组连续。

1. 90 秒错误定位

Table with 4 columns: 现象, 第一判断, 先查什么, 常见处理. Rows include: 样例 WA, 小极端 WA, 随机小数据 WA, 大数据 WA.

| 现象  | 第一判断      | 先查什么                     | 常见处理                    |
|-----|-----------|--------------------------|-------------------------|
| RE  | 越界/空容器/爆栈 | 数组大小、front/top/back、递归深度 | 加边界，改迭代或显式栈             |
| TLE | 复杂度不够     | $n*q$ 、 $n^2$ 、重复 DFS    | 换路由，先 memo / 树状数组 / BFS |

## 2. 通用 WA 检查

| 检查项     | 典型错误                         | 最小验错                |
|---------|------------------------------|---------------------|
| 题目目标    | 求最小却写最大；求方案数却输出最优值           | 手算 2-3 个元素          |
| 数据范围    | $n=2e5$ 还写双重循环               | 最大范围估复杂度            |
| 下标体系    | 数组 1-index，字符串 0-index 混用    | $l=1$ 、 $r=n$ 、子串首尾 |
| 初始化     | 最大值题 $ans=0$ ，全负数错           | 全负数组                |
| 答案位置    | 背包输出 $dp[w]$ 还是 $\max dp[j]$ | 容量不必装满              |
| 无解输出    | INF 直接打印                     | 不连通、不可达、容量不足        |
| 多组数据    | 图、memo、队列、数组未清空              | 两组数据，第二组很小          |
| 取模      | 加法/乘法中途溢出或忘 %MOD             | 答案超过 MOD            |
| 排序比较    | cmp 用 $<=$                   | 相等元素                |
| 图方向     | 有向边当无向，或反过来                  | 2 点 1 条单向边          |
| 边权      | 有负边还用 Dijkstra               | 3 点负边反例             |
| 重复值/重边  | 压缩、最短路、MST 未处理               | 两条不同权重重边            |
| DP 循环顺序 | 0/1 背包正序导致重复选                | 一个物品被选两次            |

## 3. 通用 RE 检查

| 检查项         | 典型错误                                  | 快速修法                      |
|-------------|---------------------------------------|---------------------------|
| 数组大小        | 开 $n$ 却访问 $n$ 或 $n+1$                 | 1-index 开 $n+2$           |
| 差分          | $diff[r+1]$ 越界                        | 开 $n+2$ ，或判断 $r < n$      |
| 树状数组        | $add(0, x)$ 死循环                       | 压缩 id 从 1 开始              |
| 空容器         | $q.front()$ / $st.top()$ / $v.back()$ | 访问前判 $empty()$            |
| 递归深度        | 链状树 DFS 爆栈                            | 改迭代或确认平台栈                 |
| 位运算         | $1 \ll n$ 当 $n \geq 31$               | 用 $1LL \ll n$ ，且 $n$ 不要太大 |
| 除法取模        | 除以 0 或逆元不存在                           | 先判分母                      |
| vector 尺寸   | 负数或过大转成 unsigned                      | 检查输入和内存                   |
| Floyd/DP 内存 | $5000*5000$ 11 爆内存                    | 换算法或滚动数组                  |

## 4. 通用 TLE 检查

| 检查项      | 危险写法                                 | 升级方向                                     |
|----------|--------------------------------------|------------------------------------------|
| 区间查询     | 每问循环 $[1, r]$ ， $nq$                 | PrefixSum / 树状数组 / SegmentTree           |
| 区间修改     | 每次循环加                                | Difference / LazySegmentTree             |
| DFS      | 重复状态未缓存                              | memo / DP / BFS                          |
| 最短路      | $n$ 次 Dijkstra 或 Floyd 用在大图          | 按询问数量重选模型                                |
| Dijkstra | 堆里旧状态不跳过                             | <code>if (du != dist[u]) continue</code> |
| 字符串匹配    | 每个位置重新比较模式串                          | KMP / Z / Hash                           |
| map 太慢   | 状态很多还用 <code>map&lt;tuple&gt;</code> | 数组 memo 或安全编码                            |
| 输出太慢     | 每次 endl 刷新                           | 用 <code>'\n'</code>                      |

## 5. 提交前 7 问

1. 我这版能过哪些数据范围？如果不能满分，部分分边界在哪里？
2. 是否看清多组数据，所有全局容器是否清空？
3. 所有答案、距离、方案数、乘法中间量是否用 long long？
4. 是否处理  $n=1$ 、空/无解、全相等、负数、重复值？
5. 图题是否确认方向、边权符号、是否连通、是否有重边自环？
6. DP 是否确认初始化、循环顺序、答案位置、不可达值？
7. 优化版不稳时，是否保留了能拿小数据的暴力/记忆化版本？

6. 失败后的回退路线

WA：  
小数据先写暴力对拍；没有暴力就手造极端样例。  
RE：  
先关掉优化和递归深处，查越界、空容器、多测清空。  
TLE：  
先确认复杂度是否根本不够；不够就按 ROUTE-00 重路由。  
考场口令：  
先保分，再修正解。  
如果高级版 10 分钟查不出错，先交低版本。

TRAIN-01 第 7 卷：极端样例与反例库

模块编号: TRAIN-01  
模块名称: 极端样例与反例库  
标签: 反例、极端样例、图论、DP、数据结构、字符串、数学  
一句话用途: 不会构造测试时，从本表挑 3-5 个最容易打爆错误代码的样例形状。  
题面触发词: 构造数据、自测、边界、样例太弱、怀疑 WA、提交前验错。  
什么时候用: 写完代码后；算法升级后；样例过了但担心隐藏点时。  
不要什么时候用: 不要只测极端形状而不测题面样例；不要用随机数据替代手工反例。  
复杂度: 构造样例本身无复杂度。  
数据范围参考: 所有题目，尤其是最大值、最小值、空/无解、多组数据。  
依赖的标准容器: Array、Graph、Query、State、Compressor。  
输入如何整理: 按题型挑样例形状，尽量让样例小到能手算。  
接口: 反例选择表。  
输出能力: 暴露边界、模型误判、初始化、下标、循环顺序、溢出等错误。  
下游可接: TRAIN-00 自检、TRAIN-02 路由训练。  
可拼接模块: CPP-009、BRUTE-15、ROUTE-01、图论/DP/数据结构/字符串数学模块。  
模板代码: 无。  
调用示例: 区间题至少测  $n=1$ 、 $l=1/r=n$ 、连续修改同一点、负数；图题至少测不可达、重边、自环或方向。  
常见坑: 只测“正常样例”；只测最大数据不手算；反例太大导致自己也算不清。  
暴力/部分分替代: 小规模用暴力输出作标准答案；大规模只测运行时间和溢出。  
升级方向: 把常用反例变成固定本地输入；优化版和暴力版对拍 100 组小数据。  
最小测试样例: 每类题至少准备 1 个能手算的 3-5 元素样例。

1. 极端样例构造表

| 样例形状           | 适用题型           | 能抓出的错误            |
|----------------|----------------|-------------------|
| $n=1$ / 单点     | 所有题            | 初始化、边界、空转移、 $l=r$ |
| $n=0$ 或空集合可出现  | 计数、集合、字符串      | 空状态、无解输出、越界       |
| 全相等            | 排序、去重、压缩、单调结构  | $>/>=$ 写反, 重复值处理  |
| 严格递增           | LIS、单调栈、逆序对、窗口 | 边界贡献、答案极端小/大      |
| 严格递减           | LIS、单调栈、逆序对、窗口 | 栈队弹出条件、逆序对最大      |
| 全负数            | 最大子段和、DP 最大值   | 初值错误为 0           |
| 含 0            | 数学、背包、最短路      | 除 0、0 权边、容量为 0    |
| 值接近 $1e9/1e18$ | 数学、图权、答案统计     | int 溢出、中间乘法溢出     |
| 重复值很多          | 压缩、排名、哈希、排序    | 去重、稳定性、计数         |
| 无解/不可达         | 图、DP、二分答案      | INF 直接输出、误判可行     |

| 样例形状                             | 适用题型        | 能抓出的错误           |
|----------------------------------|-------------|------------------|
| 多组数据两组                           | 所有题         | 未清空数组、图、memo、队列  |
| 查询边界 $[1,1]$ 、 $[1,n]$ 、 $[n,n]$ | 区间题         | 前缀、差分、线段树下标      |
| 所有操作都修改，无查询                      | 数据结构        | 输出时机、懒标记残留       |
| 所有操作都查询，无修改                      | 数据结构        | 初始 build、空操作     |
| 链状图/链状树                          | 图、树 DP、LCA  | DFS 深度、父子方向、路径边界 |
| 星状图/星状树                          | 图、树         | 根、深度、多个儿子合并      |
| 不连通图                             | BFS、MST、最短路 | 可达性、MST 无解       |
| 重边、自环                            | 图           | 取 min、DSU、最短路松弛  |
| $k=1$ / $k=n$                    | 窗口、选择、分组    | 窗口出队、边界答案        |
| 模数附近                             | 计数、组合、快速幂   | 忘取模、负数取模         |

## 2. 图论反例

| 错误模型         | 最小反例形状                                                               | 应该提醒自己                            |
|--------------|----------------------------------------------------------------------|-----------------------------------|
| 用 DFS 求无权最短路 | 1 到 3 有直边，1 到 2 到 3 也可达                                              | 最少步数必须 BFS                        |
| 有权图用 BFS     | 两条边数少但权重大，一条边数多但权小                                                   | BFS 只适合等权                         |
| 负边用 Dijkstra | $1 \rightarrow 2(2)$ ， $1 \rightarrow 3(5)$ ， $3 \rightarrow 2(-10)$ | 有负边换 Bellman-Ford/SPFA 或重路由       |
| 把有向图当无向      | 只有 $1 \rightarrow 2$ ，问 2 到 1                                        | 建图方向必须按题面                         |
| 以为是树但不是      | $m \neq n-1$ 或不连通                                                    | LCA/树 DP 前先确认树                    |
| MST 忘判连通     | 4 点只有两条边                                                             | $\text{cnt} == n-1$ 才有生成树         |
| Topo 不判环     | $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$                        | $\text{order.size()} == n$ 才是 DAG |
| Floyd INF 溢出 | $\text{dist}[i][k] = \text{INF}$ 还相加                                 | 加前判断两段都可达                         |
| 二分图只测连通块 1   | 两个连通块，第二个是奇环                                                         | 每个未染色点都 BFS                       |
| 重边不取最优       | 1-2 有权 10 和 1                                                        | 最短路/Floyd 初始化取 min                |

## 3. DP 反例

| 错误模型        | 最小反例形状                                                | 应该提醒自己                        |
|-------------|-------------------------------------------------------|-------------------------------|
| 0/1 背包容量正序  | 一个物品 $w=1, v=1, W=2$                                  | 0/1 必须倒序，防重复选                 |
| 完全背包容量倒序    | 一个物品可无限选                                              | 完全背包通常正序                      |
| 最大值初值为 0    | 所有收益为负但必须选                                            | 最大值可能为负，用 $-\text{INF}$       |
| 可行性和最优值混用   | 不可达状态参与转移                                             | 先判断不是 INF                     |
| 答案位置错       | 容量不要求装满                                               | 可能要 $\max dp[j]$ 而不是 $dp[W]$  |
| 状态漏 last    | 相同 $i, \text{rest}$ ，上一步不同影响限制                        | 影响未来的量必须进状态                   |
| memo 遇到有环   | $\text{dfs}(u)$ 可回到 $u$                               | 用 visiting 检环或改图模型            |
| 区间 DP 长度顺序错 | $\text{dp}[l][r]$ 依赖更短区间                              | 按 len 从小到大                    |
| LCS 下标错     | 空前缀没有初始化                                              | 开 $n+1, m+1$ ，从 1 映射字符        |
| 状压溢出        | $n=20$ 写 $\text{int total}=1 \ll n$ 还好， $n \geq 31$ 爆 | 用 $1 \text{LL} \ll n$ ，并确认复杂度 |

## 4. 数据结构反例

| 错误模型              | 最小反例形状                                  | 应该提醒自己                        |
|-------------------|-----------------------------------------|-------------------------------|
| 树状数组下标从 0 开始      | 压缩 id 返回 0                              | 树状数组必须 1-index                |
| 单点赋值当作加法          | 初值 5，赋成 7，却 $\text{add}(\text{pos}, 7)$ | 赋值要加 new-old                  |
| 差分 $r=n$ 越界       | 区间 $[1,n]$ 加                            | diff 开 $n+2$                  |
| 前缀和没处理 $l=1$      | 查询 $[1,r]$                              | $\text{sum}[0]=0$             |
| 线段树懒标记不下传         | 先整段加，再查子段                               | 查询/递归前 push                   |
| Sparse Table 做区间和 | 重叠区间相加出错                                | ST 适合 min/max/gcd，不适合 sum     |
| 单调队列忘弹窗口外         | $k=2$ ，队首过期                             | 每步先/后按统一规则弹过期                 |
| 单调栈等号写错           | 全相等数组                                   | 先确定要“严格”还是“不严格”               |
| Compressor 查询不存在值 | 问区间 $[L,R]$ 但 $L/R$ 不在数组                | 用 lower_bound/upper_bound 找范围 |
| DSU 多测未 init      | 第二组数据点数更小                               | 每组 $\text{init}(n)$           |

## 5. 字符串 / 数学反例

| 错误模型        | 最小反例形状               | 应该提醒自己                              |
|-------------|----------------------|-------------------------------------|
| 字符串下标混用     | 模式在第 1 个字符出现         | 字符串默认 0-index                       |
| KMP 漏重叠匹配   | 文本 aaaaa, 模式 aaa     | 匹配后回退到 $pi[m-1]$                    |
| Hash 不处理负数  | 子串哈希相减为负             | $(x \% MOD + MOD) \% MOD$           |
| Hash 只用单模   | 特殊数据碰撞               | 重要题用双模或 KMP                         |
| Trie 多测未清空  | 第二组词更少               | 重置节点数组和计数                           |
| $\gcd(0,0)$ | 两数都为 0               | 按题意特判                               |
| lcm 溢出      | $a*b/g$ 先乘爆          | $a/g*b$ , 必要时 <code>__int128</code> |
| 逆元不存在       | a 与 MOD 不互质          | 费马逆元要求 MOD 质数且 a 非 0                |
| 快速幂指数为 0    | $a^0$                | 返回 1                                |
| 组合数预处理不够    | 查询 $C(n,k)$ 中 n 超过上限 | 按最大 n 预处理                           |
| 筛法上界错       | 需要判断 $\sqrt{x}$ 以内质数 | 上界至少覆盖需求                            |
| 矩阵快速幂忘单位阵   | 指数为 0                | 初始答案矩阵为 I                           |

## 6. 最小反例构造口诀

先小到能手算。

再卡边界：1、0、n、最大值。

再卡重复：重复点、重复边、重复字符。

再卡无解：不可达、容量不足、空答案。

最后卡复杂度：最大 n、最大 q、最大值域。

## TRAIN-02 第 7 卷：题面摘要路由训练

模块编号：TRAIN-02

模块名称：题面摘要路由训练

标签：路由训练、题面信号、数据范围、部分分、升级、最小验错

一句话用途：用短题面训练“看到信号 -> 判断复杂度 -> 选模块 -> 先交部分分 -> 再升级”。

题面触发词：考前训练、模拟路由、不会做正解、先拿部分分、模块组合。

什么时候用：考前每天快速刷；比赛中读完题后对照路由；训练弱基础考生建立模块反射。

不要什么时候用：不要把本模块当完整题解；这里只练判断和提交路线，不展开证明。

复杂度：每题按数据范围决定。

数据范围参考：覆盖  $n \leq 10$ 、 $n \leq 20$ 、 $n \leq 500$ 、 $n, q \leq 2e5$ 、值域  $1e9/1e18$  等常见档。

依赖的标准容器：Array、Graph、Query、State、Compressor。

输入如何整理：每题先圈数据范围，再圈操作、目标、边权、修改、是否多测。

接口：每题只给六项：题面信号、数据范围判断、优先模块、部分分版本、升级版本、最小验错。

输出能力：给出路由答案和提交策略。

下游可接：ROUTE-00、ROUTE-01、BRUTE、DP、DS、GRAPH、MATH、STR 各卷。

可拼接模块：所有模块。

模板代码：无。

调用示例：遮住“优先模块”一列，自己先判断，再对照。

常见坑：只看关键词不看数据范围；看到“最短”不看边权；看到“区间”不看有没有修改。

暴力/部分分替代：每题都列部分分版本。

升级方向：每题都列升级版本。

最小测试样例：每题都列最小验错。

## 训练使用法

每题限时 60 秒：

1. 圈信号。
2. 写目标复杂度。
3. 选优先模块。
4. 写一个能先交的部分分版本。
5. 写升级版本。
6. 写 3 个最小验错点。

### 01 静态区间和

题面信号：给定数组，多次询问  $[l, r]$  的元素和，数组不修改。

数据范围判断： $n, q \leq 2e5$ ，不能每问循环。

优先模块：DS-01 PrefixSum。

部分分版本：每次从 1 到  $r$  循环求和。

升级版本：预处理前缀和， $query(l, r) = s[r] - s[l-1]$ 。

最小验错： $l=r$ ； $l=1$ ；全负数；答案超过  $int$ 。

### 02 单点修改区间和

题面信号：两类操作，单点赋值或加值，询问区间和。

数据范围判断： $n, q \leq 2e5$ ，需要  $O(\log n)$ 。

优先模块：DS-02 树状数组。

部分分版本：数组直接改，查询循环求和。

升级版本：树状数组；赋值操作转成  $\text{delta} = \text{new} - \text{old}$ 。

最小验错：连续修改同一位置；查询单点；负数修改； $n=1$ 。

### 03 区间加最后输出

题面信号：多次对  $[l, r]$  加  $x$ ，所有操作结束后输出最终数组。

数据范围判断： $n, q \leq 2e5$ ，但没有在线查询。

优先模块：DS-01 Difference。

部分分版本：每次循环把区间内元素加上  $x$ 。

升级版本：差分  $\text{diff}[l] += x$ ， $\text{diff}[r+1] -= x$ ，最后前缀还原。

最小验错： $r=n$ ；负数加；区间长度 1；所有区间都覆盖。

### 04 大坐标排名统计

题面信号：值域到  $1e9/1e18$ ，需要统计小于等于某值的个数或动态排名。

数据范围判断：操作数  $\leq 2e5$ ，值域大但出现次数有限。

优先模块：CPP-007 Compressor + DS-02 树状数组。

部分分版本：维护  $vector$ ，每次查询循环或排序。

升级版本：离线收集坐标，压缩后用树状数组维护频次。

最小验错：重复值；负坐标；查询范围内没有数；边界值没出现。

### 05 逆序对

题面信号：求  $i < j$  且  $a[i] > a[j]$  的数量，或交换排序次数。

数据范围判断： $n \leq 2e5$ ，双重循环只能拿小数据。

优先模块：CPP-007 Compressor + DS-02 树状数组。

部分分版本：双重循环统计。

升级版本：从左到右扫描，统计前面比当前大的数。

最小验错：全相等；严格递增；严格递减；答案用 long long。

## 06 无权网格最少步数

题面信号：网格有障碍，从起点到终点，每步代价相同，求最少步数。

数据范围判断： $n*m \leq 2e5$  或  $1000*1000$ ，需要线性。

优先模块：GRAPH-02 BFS + queue。

部分分版本：DFS 只判可达或大图枚举。

升级版本：BFS, dist 初始化 -1，首次到达即最短。

最小验错：起点等于终点；不可达；一行/一列；障碍在边界。

## 07 非负权单源最短路

题面信号：图有边权，权值非负，从一个起点到各点最小花费。

数据范围判断： $n, m \leq 2e5$ ，需要邻接表和堆。

优先模块：GRAPH-03 Dijkstra + CPP-004 priority\_queue。

部分分版本：Bellman-Ford 过大数据，或大图 DFS 枚举简单路径。

升级版本：Dijkstra，弹出堆顶时跳过旧状态。

最小验错：不可达；0 权边；重边；题面是否真的无负边。

## 08 有负边小图最短路

题面信号：边权可能为负，问从起点到终点最短距离，可能需要判断负环。

数据范围判断： $n \leq 500, m \leq 5000$  可考虑  $O(nm)$ ；大图要谨慎。

优先模块：GRAPH-04 Bellman-Ford / SPFA。

部分分版本：若无负边子任务，用 Dijkstra；极小图枚举路径。

升级版本：Bellman-Ford 松弛  $n-1$  轮，额外一轮判负环。

最小验错：负边但无负环；不可达；负环不影响终点；重边。

## 09 最小生成树

题面信号：无向边连接城市，要求总费用最小让所有点连通。

数据范围判断： $n, m \leq 2e5$ ，排序边可接受。

优先模块：GRAPH-06 DSU + Kruskal。

部分分版本：只判连通或大数据枚举边集。

升级版本：边按权排序，能合并就加入，最后检查边数  $n-1$ 。

最小验错：图不连通；重边；负权边；只有一个点。

## 10 DAG 任务依赖

题面信号：任务有先后依赖，求合法顺序、最长链或方案数。

数据范围判断： $n, m \leq 2e5$ ，需要  $O(n+m)$ 。

优先模块：GRAPH-05 Topo + DAG DP。

部分分版本：DFS + memo，小数据递归求；或只输出一个可行顺序。

升级版本：拓扑排序后按 order 转移 dp。

最小验错：多个入度 0；有环；不连通 DAG；方案数取模。



## 11 树上路径距离

题面信号:  $n$  个点  $n-1$  条边, 多次询问树上两点距离或 LCA。

数据范围判断:  $q \leq 2e5$ , 每问 DFS 会超时。

优先模块: GRAPH-09 LCA。

部分分版本: 每次从  $u$  DFS/BFS 到  $v$ 。

升级版本: 预处理倍增 LCA, 距离用根距相减。

最小验错:  $u=v$ ; 链状树; 星状树; 根参与查询。

## 12 0/1 背包

题面信号: 每个物品最多选一次, 有重量和价值, 容量限制。

数据范围判断: 若  $n \cdot W \leq 1e7$  可 DP;  $n \leq 25$  可 DFS 部分分。

优先模块: DP-06 0/1 Knapsack。

部分分版本: DFS 枚举每个物品选或不选。

升级版本: 一维  $dp[j]$ , 容量从大到小枚举。

最小验错:  $W=0$ ; 物品重量超过  $W$ ; 一个物品不能选两次; 价值为 0。

## 13 完全背包

题面信号: 每种物品可选无限次, 在容量内最大价值或方案数。

数据范围判断:  $n \cdot W$  可承受时直接 DP。

优先模块: DP-07 Complete Knapsack。

部分分版本: DFS 枚举每种选几个, 小数据可过。

升级版本: 一维  $dp[j]$ , 容量从小到大枚举。

最小验错: 容量为 0; 只有一种物品; 循环方向; 方案数是否取模。

## 14 两个字符串匹配

题面信号: 两个字符串, 求最长公共子序列或最少编辑次数。

数据范围判断:  $n, m \leq 3000$  可  $O(nm)$ ; 更大要换路由。

优先模块: DP-09 LCS / DP-10 EditDistance。

部分分版本: 递归搜索或只处理很短字符串。

升级版本: 二维 DP, 开  $n+1$  和  $m+1$  处理空前缀。

最小验错: 空前缀; 完全相同; 完全不同; 重复字符。

## 15 区间合并

题面信号: 石子合并、括号匹配、删除区间, 转移需要枚举分割点。

数据范围判断:  $n \leq 300/500$  可  $O(n^3)$ 。

优先模块: DP-13 Interval DP + DS-01 PrefixSum。

部分分版本: DFS 枚举合并顺序 + memo。

升级版本: 按区间长度从小到大表推, 枚举  $k$ 。

最小验错: 长度 1; 长度 2; INF 初始化; 区间和下标。

## 16 访问所有关键点

题面信号:  $n \leq 20$ , 要求访问每个点一次或至少一次, 求最小代价。

数据范围判断: 集合相关且  $n$  小,  $2^n \cdot n^2$  可接受。

优先模块: GRAPH-03/04 shortest path + DP-16 Bitmask DP。

部分分版本: 全排列枚举访问顺序。

升级版本:  $dp[mask][last]$  枚举下一个未访问点。

最小验错: 关键点不可达; 起点是否固定; 是否要回起点;  $1 < n$ 。

## 17 单模式字符串查找

题面信号: 在长文本中多次查找模式串出现位置或出现次数。

数据范围判断: 文本和模式总长  $\leq 1e6$ , 不能朴素逐位比较。

优先模块: STR-02 KMP / Z Function。

部分分版本: 每个位置暴力比较模式串。

升级版本: KMP 前缀函数, 匹配后处理重叠出现。

最小验错: 模式在开头; 模式在结尾; 重叠匹配; 模式比文本长。

## 18 组合数取模

题面信号: 多次询问  $C(n, k) \bmod p$ ,  $p$  为质数,  $n$  上限固定。

数据范围判断: 查询多, 最大  $n$  可预处理阶乘和逆元。

优先模块: MATH-03 Factorial + Modular Inverse + Combination。

部分分版本: 按定义循环计算小  $n$ 。

升级版本: 预处理  $fac/ifac$ , 每问  $O(1)$ 。

最小验错:  $k=0$ ;  $k=n$ ;  $k>n$ ;  $n$  等于预处理上界。

## v0.2 本卷例题训练区

这一节是 0.2 新增的实战例题。每题都配完整可运行代码和样例; 考试时优先看“覆盖模块”和“考场用途”, 再复制对应代码骨架。

### V07-EX01 多组图连通块清空

- 归属卷: 第 7 卷
- 覆盖模块: TRAIN-00、TRAIN-01、GRAPH-01
- 考场用途: 训练多组数据时每组重新建图、清空 `visited` 和邻接表。

**题目描述:** 给定  $T$  组无向图, 分别输出每组图连通块个数。

**输入格式:** 第一行一个整数  $T$ 。每组第一行两个整数  $n\ m$ , 接下来  $m$  行每行一条无向边  $u\ v$ 。

**输出格式:** 每组输出一行连通块个数。

**样例输入:**

```
2
4 2
1 2
3 4
3 0
```

**样例输出:**

```
2
3
```

**完整代码:**

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n, m;
        cin >> n >> m;
        vector<vector<int>> graph(n + 1);
        for (int i = 1; i <= m; i++) {
            int u, v;
            cin >> u >> v;
            graph[u].push_back(v);
            graph[v].push_back(u);
        }
        vector<int> visited(n + 1, 0);
        int components = 0;
        for (int start = 1; start <= n; start++) {
            if (visited[start]) continue;
            components++;
            queue<int> q;
            q.push(start);
            visited[start] = 1;
            while (!q.empty()) {
                int u = q.front();
                q.pop();
                for (int v : graph[u]) {
                    if (!visited[v]) {
                        visited[v] = 1;
                        q.push(v);
                    }
                }
            }
        }
        cout << components << '\n';
    }
    return 0;
}

```

#### 测试设计:

- 测试 1 输入:

```

1
1 0

```

期望输出:

```

1

```

- 测试 2 输入:

```

2
3 2
1 2
2 3
2 0

```

期望输出:

```

1
2

```

## V07-EX02 前缀和暴力核验

- 归属卷：第 7 卷
- 覆盖模块：TRAIN-00、TRAIN-02、DS-01
- 考场用途：训练优化版写完保留暴力版，对每个查询做小规模核验。

**题目描述：**给定数组和若干区间和询问。程序同时用前缀和与暴力循环计算答案；若发现不一致，输出 CHECK\_FAILED 并结束，否则输出每次询问答案。

**输入格式：**第一行两个整数  $n$   $q$ 。第二行  $n$  个整数。接下来  $q$  行，每行两个整数  $l$   $r$ 。

**输出格式：**若核验通过，每个询问输出一行答案；否则输出 CHECK\_FAILED。

**样例输入：**

```
4 3
2 -1 5 3
1 4
2 3
4 4
```

**样例输出：**

```
9
4
3
```

**完整代码：**

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;
    vector<ll> a(n + 1), prefix(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        prefix[i] = prefix[i - 1] + a[i];
    }
    while (q--) {
        int l, r;
        cin >> l >> r;
        ll fast = prefix[r] - prefix[l - 1];
        ll slow = 0;
        for (int i = l; i <= r; i++) slow += a[i];
        if (fast != slow) {
            cout << "CHECK_FAILED\n";
            return 0;
        }
        cout << fast << '\n';
    }
    return 0;
}
```

**测试设计：**

- 测试 1 输入：

```
1 1
-8
1 1
```

**期望输出：**

```
-8
```

- 测试 2 输入:

```
3 2
10 20 30
1 1
1 3
```

期望输出:

```
10
60
```

### V07-EX03 背包循环顺序反例

- 归属卷: 第 7 卷
- 覆盖模块: TRAIN-01、DP-06、BRUTE-15
- 考场用途: 训练用最小反例识别 0/1 背包容量正序导致重复选择的问题。

**题目描述:** 给定 0/1 背包数据。程序输出正确答案, 并判断“错误的容量正序写法”是否会得到不同结果。若不同, 第二行输出 LOOP\_RISK, 否则输出 SAME。

**输入格式:** 第一行两个整数  $n$   $W$ 。接下来  $n$  行, 每行  $w$   $v$ 。

**输出格式:** 第一行输出 0/1 背包正确最大价值。第二行输出 LOOP\_RISK 或 SAME。

**样例输入:**

```
1 2
1 1
```

**样例输出:**

```
1
LOOP_RISK
```

**完整代码:**

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, W;
    cin >> n >> W;
    vector<int> w(n + 1);
    vector<ll> v(n + 1);
    for (int i = 1; i <= n; i++) cin >> w[i] >> v[i];

    vector<ll> correct(W + 1, 0), forward_wrong(W + 1, 0);
    for (int i = 1; i <= n; i++) {
        for (int cap = W; cap >= w[i]; cap--) {
            correct[cap] = max(correct[cap], correct[cap - w[i]] + v[i]);
        }
        for (int cap = w[i]; cap <= W; cap++) {
            forward_wrong[cap] = max(forward_wrong[cap], forward_wrong[cap - w[i]] + v[i]);
        }
    }

    cout << correct[W] << '\n';
    cout << (correct[W] == forward_wrong[W] ? "SAME" : "LOOP_RISK") << '\n';
    return 0;
}
```

**测试设计:**

- 测试 1 输入:

```
2 3
2 5
3 7
```

期望输出:

```
7
SAME
```

- 测试 2 输入:

```
1 3
1 2
```

期望输出:

```
2
LOOP_RISK
```

#### V07-EX04 最短路双算法核验

- 归属卷: 第 7 卷
- 覆盖模块: TRAIN-00、GRAPH-03、GRAPH-04
- 考场用途: 训练用慢速 Bellman-Ford 核验 Dijkstra, 抓旧堆状态、重边和不可达输出问题。

**题目描述:** 给定无向非负权图和起点  $s$ 。程序分别用 Dijkstra 和 Bellman-Ford 求最短路; 若结果不一致, 输出 CHECK\_FAILED, 否则输出从  $s$  到每个点的最短距离, 不可达输出 -1。

**输入格式:** 第一行三个整数  $n$   $m$   $s$ 。接下来  $m$  行, 每行  $u$   $v$   $w$ 。

**输出格式:** 输出一行  $n$  个整数, 表示到  $1..n$  的距离。

**样例输入:**

```
4 4 1
1 2 10
1 2 3
2 3 4
4 4 0
```

**样例输出:**

```
0 3 7 -1
```

**完整代码:**

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = 4'000'000'000'000'000'000LL;

struct Edge {
    int u;
    int v;
    ll w;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, s;
    cin >> n >> m >> s;
    vector<vector<pair<int, ll>>> graph(n + 1);
    vector<Edge> edges;
    for (int i = 1; i <= m; i++) {
        int u, v;
        ll w;
        cin >> u >> v >> w;
        graph[u].push_back({v, w});
        graph[v].push_back({u, w});
    }
```

```

        edges.push_back({u, v, w});
        edges.push_back({v, u, w});
    }

    vector<ll> dijkstra(n + 1, INF);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
    dijkstra[s] = 0;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [du, u] = pq.top();
        pq.pop();
        if (du != dijkstra[u]) continue;
        for (auto [v, w] : graph[u]) {
            if (dijkstra[v] > du + w) {
                dijkstra[v] = du + w;
                pq.push({dijkstra[v], v});
            }
        }
    }

    vector<ll> bellman(n + 1, INF);
    bellman[s] = 0;
    for (int round = 1; round <= n - 1; round++) {
        bool changed = false;
        for (const Edge &e : edges) {
            if (bellman[e.u] == INF) continue;
            if (bellman[e.v] > bellman[e.u] + e.w) {
                bellman[e.v] = bellman[e.u] + e.w;
                changed = true;
            }
        }
        if (!changed) break;
    }

    for (int i = 1; i <= n; i++) {
        if (dijkstra[i] != bellman[i]) {
            cout << "CHECK_FAILED\n";
            return 0;
        }
    }
    for (int i = 1; i <= n; i++) {
        if (i > 1) cout << ' ';
        cout << (dijkstra[i] == INF ? -1 : dijkstra[i]);
    }
    cout << '\n';
    return 0;
}

```

#### 测试设计:

- 测试 1 输入:

```

3 1 1
1 2 5

```

期望输出:

```
0 5 -1
```

- 测试 2 输入:

```

3 3 1
1 2 0
2 3 2
1 3 5

```

期望输出:

## V07-EX05 KMP 与暴力重叠核验

- 归属卷：第 7 卷
- 覆盖模块：TRAIN-00、STR-02、CPP-011
- 考场用途：训练字符串算法用暴力版核验，重点覆盖重叠匹配。

**题目描述：**给定文本串  $s$  和模式串  $p$ 。程序用 KMP 和暴力匹配分别统计出现次数；若不一致，输出 CHECK\_FAILED，否则输出出现次数。

**输入格式：**第一行字符串  $s$ 。第二行字符串  $p$ 。

**输出格式：**输出核验后的出现次数，或 CHECK\_FAILED。

**样例输入：**

```
aaaaa
aa
```

**样例输出：**

```
4
```

**完整代码：**

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s_raw, p_raw;
    cin >> s_raw >> p_raw;
    string s = " " + s_raw;
    string p = " " + p_raw;
    int n = (int)s_raw.size();
    int m = (int)p_raw.size();

    vector<int> pi(m + 1, 0);
    for (int i = 2; i <= m; i++) {
        int j = pi[i - 1];
        while (j > 0 && p[i] != p[j + 1]) j = pi[j];
        if (p[i] == p[j + 1]) j++;
        pi[i] = j;
    }

    int kmp_count = 0;
    int j = 0;
    for (int i = 1; i <= n; i++) {
        while (j > 0 && s[i] != p[j + 1]) j = pi[j];
        if (s[i] == p[j + 1]) j++;
        if (j == m) {
            kmp_count++;
            j = pi[j];
        }
    }

    int brute_count = 0;
    for (int i = 1; i + m - 1 <= n; i++) {
        bool ok = true;
        for (int t = 1; t <= m; t++) {
            if (s[i + t - 1] != p[t]) ok = false;
        }
        if (ok) brute_count++;
    }

    if (kmp_count != brute_count) {
```



```

        cout << "CHECK_FAILED\n";
    } else {
        cout << kmp_count << '\n';
    }
    return 0;
}

```

#### 测试设计:

- 测试 1 输入:

abababa  
aba

期望输出:

3

- 测试 2 输入:

abcde  
f

期望输出:

0

#### V07-EX06 空栈保护括号匹配

- 归属卷: 第 7 卷
- 覆盖模块: TRAIN-00、CPP-004、CPP-009
- 考场用途: 训练访问 top 前先判空, 避免空容器运行错误。

**题目描述:** 给定 T 个只含括号字符的字符串, 判断每个字符串是否合法。括号包含 ()[]{} 三种, 必须正确嵌套。

**输入格式:** 第一行一个整数 T。接下来 T 行, 每行一个字符串。

**输出格式:** 每个字符串输出 YES 或 NO。

**样例输入:**

```

3
([ ])
([ ])
)(

```

**样例输出:**

```

YES
NO
NO

```

**完整代码:**

```

#include <bits/stdc++.h>
using namespace std;

bool match(char left, char right) {
    return (left == '(' && right == ')') ||
           (left == '[' && right == ']') ||
           (left == '{' && right == '}');
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        string s;
        cin >> s;
    }
}

```

```

    stack<char> st;
    bool ok = true;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') {
            st.push(c);
        } else {
            if (st.empty() || !match(st.top(), c)) {
                ok = false;
                break;
            }
            st.pop();
        }
    }
    if (!st.empty()) ok = false;
    cout << (ok ? "YES" : "NO") << '\n';
}
return 0;
}

```

#### 测试设计:

- 测试 1 输入:

```

2
()
(

```

期望输出:

```

YES
NO

```

- 测试 2 输入:

```

2
]
{[]

```

期望输出:

```

NO
YES

```

#### V07-EX07 乘法溢出上限判断

- 归属卷: 第 7 卷
- 覆盖模块: TRAIN-00、CPP-008、MATH-02
- 考场用途: 训练大数乘法比较时使用 `__int128`, 避免 `long long` 中间乘法溢出。

**题目描述:** 给定  $q$  个询问, 每个询问包含非负整数  $a$   $b$   $limit$ 。判断  $a*b$  是否不超过  $limit$ 。

**输入格式:** 第一行一个整数  $q$ 。接下来  $q$  行, 每行三个整数  $a$   $b$   $limit$ 。

**输出格式:** 每个询问输出 YES 或 NO。

**样例输入:**

```

3
3 4 12
3 5 14
1000000000000 1000000000000 1000000000000000000

```

**样例输出:**

```

YES
NO
NO

```

**完整代码:**

```

#include <bits/stdc++.h>
using namespace std;

```

```

using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    cin >> q;
    while (q--) {
        ll a, b, limit;
        cin >> a >> b >> limit;
        __int128 product = (__int128)a * b;
        cout << (product <= limit ? "YES" : "NO") << '\n';
    }
    return 0;
}

```

#### 测试设计:

- 测试 1 输入:

```

2
0 999999999999999999 0
1 999999999999999999 999999999999999999

```

期望输出:

```

YES
YES

```

- 测试 2 输入:

```

1
3037000500 3037000500 9223372036854775807

```

期望输出:

```

NO

```

#### V07-EX08 二分答案边界核验

- 归属卷: 第 7 卷
- 覆盖模块: TRAIN-02、ROUTE-00、GREEDY-02
- 考场用途: 训练二分答案的左右边界、check 单调性和  $k=1$ 、 $k=n$  反例。

**题目描述:** 给定  $n$  个正整数任务量, 按原顺序分成不超过  $k$  个连续段。最小化所有段和的最大值。

**输入格式:** 第一行两个整数  $n$   $k$ 。第二行  $n$  个正整数。

**输出格式:** 输出最小可能的最大段和。

**样例输入:**

```

5 2
7 2 5 10 8

```

**样例输出:**

```

18

```

**完整代码:**

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;

bool can_split(const vector<ll> &a, int n, int k, ll limit) {
    int groups = 1;
    ll current = 0;
    for (int i = 1; i <= n; i++) {
        if (a[i] > limit) return false;
        if (current + a[i] <= limit) {

```

```

        current += a[i];
    } else {
        groups++;
        current = a[i];
    }
}
return groups <= k;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    cin >> n >> k;
    vector<ll> a(n + 1);
    ll left = 0, right = 0;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        left = max(left, a[i]);
        right += a[i];
    }
    while (left < right) {
        ll mid = left + (right - left) / 2;
        if (can_split(a, n, k, mid)) right = mid;
        else left = mid + 1;
    }
    cout << left << '\n';
    return 0;
}

```

#### 测试设计:

- 测试 1 输入:

```

3 1
1 2 3

```

期望输出:

```

6

```

- 测试 2 输入:

```

3 3
1 2 3

```

期望输出:

```

3

```

#### V07-EX09 RMQ 暴力对拍

- 归属卷: 第 7 卷
- 覆盖模块: TRAIN-00、DS-03、TRAIN-01
- 考场用途: 训练静态区间最小值用 Sparse Table, 并用暴力循环核验边界。

**题目描述:** 给定数组和若干区间最小值询问。程序用 Sparse Table 和暴力分别计算; 若不一致输出 CHECK\_FAILED, 否则输出每个询问的最小值。

**输入格式:** 第一行两个整数  $n$   $q$ 。第二行  $n$  个整数。接下来  $q$  行, 每行 1  $r$ 。

**输出格式:** 每个询问输出一行最小值, 或输出 CHECK\_FAILED。

**样例输入:**

```

5 3
4 2 7 1 3
1 5

```

2 3  
4 4

样例输出:

1  
2  
1

完整代码:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];

    vector<int> lg(n + 1, 0);
    for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
    int K = lg[n] + 1;
    vector<vector<int>> st(K, vector<int>(n + 1));
    for (int i = 1; i <= n; i++) st[0][i] = a[i];
    for (int k = 1; k < K; k++) {
        for (int i = 1; i + (1 << k) - 1 <= n; i++) {
            st[k][i] = min(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
        }
    }

    while (q--) {
        int l, r;
        cin >> l >> r;
        int len = r - l + 1;
        int k = lg[len];
        int fast = min(st[k][l], st[k][r - (1 << k) + 1]);
        int slow = a[l];
        for (int i = l; i <= r; i++) slow = min(slow, a[i]);
        if (fast != slow) {
            cout << "CHECK_FAILED\n";
            return 0;
        }
        cout << fast << '\n';
    }
    return 0;
}
```

测试设计:

- 测试 1 输入:

1 1  
-5  
1 1

期望输出:

-5

- 测试 2 输入:

4 2  
8 8 8 8

1 4  
2 2

期望输出:

8  
8

### V07-CEX01 快慢算法核验逆序对

- 归属卷: 第 7 卷
- 覆盖模块: 对拍、暴力核验
- 考场用途: 写完高级算法先用慢算法对照小数据。
- 参考题型来源: 参考来源: 竞赛对拍常规做法。

**题目描述:** 同时用  $O(n^2)$  和树状数组算逆序对, 输出是否一致。

**输入格式:** 第一行  $n$ , 第二行数组。

**输出格式:** 输出 OK/BAD 和答案。

**样例输入:**

5  
3 1 2 5 4

**样例输出:**

OK 3

**完整代码:**

```
#include <bits/stdc++.h>
using namespace std;
```

```
long long slow(vector<int>a){long long ans=0;for(int i=0;i<(int)a.size();i++)for(int j=i+1;j<(int)a.size();j++)if(a[i]>a[j])ans++;}
long long fast(vector<int>a){int n=a.size();vector<int>xs=a;sort(xs.begin(),xs.end());xs.erase(unique(xs.begin(),xs.end()),xs.end());}
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;cin>>n;vector<int>a(n);for(int&i:a)cin>>i;cout<<(slow(a)-fast(a))<<endl;}
```

**测试设计:** 额外测试: 构造最小规模、重复值、边界值各一组, 和样例一起运行。

---

### V07-CEX02 输入范围守卫

- 归属卷: 第 7 卷
- 覆盖模块: 边界检查、调试
- 考场用途: 本地调试时先检查数据是否满足题面。
- 参考题型来源: 参考来源: 调试训练经验。

**题目描述:** 检查数组元素是否都在  $[-1e9, 1e9]$ 。

**输入格式:** 第一行  $n$ , 第二行数组。

**输出格式:** 输出检查结果。

**样例输入:**

3  
1 -2 1000000001

**样例输出:**

INPUT\_OUT\_OF\_RANGE

**完整代码:**

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;cin>>n;vector<long long>a(n+1);for(int i=1;i<=n;i++)cin>>a[i];}
```

**测试设计:** 额外测试: 构造最小规模、重复值、边界值各一组, 和样例一起运行。

---

### V07-CEX03 二分边界可视化

- 归属卷：第 7 卷
- 覆盖模块：二分答案、边界
- 考场用途：用最小满足模型避免死循环。
- 参考题型来源：参考来源：二分答案常见错误清单。

**题目描述：**求最小  $x$ ，使  $x^2 \geq \text{target}$ ，范围 1..100。

**输入格式：**输入 target。

**输出格式：**输出  $x$ 。

**样例输入：**

50

**样例输出：**

8

**完整代码：**

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);long long l=1,r=100,ans=-1,target;cin>>target;while(l<=r){l
```

**测试设计：**额外测试：构造最小规模、重复值、边界值各一组，和样例一起运行。

---

### V07-CEX04 乘法溢出探针

- 归属卷：第 7 卷
- 覆盖模块：\_\_int128、防溢出
- 考场用途：判断乘积时不要先溢出。
- 参考题型来源：参考来源：数值边界调试经验。

**题目描述：**判断  $a*b$  是否超过 limit。

**输入格式：**输入 a b limit。

**输出格式：**输出 OK 或 OVER。

**样例输入：**

1000000000000 1000000000000 1000000000000000000

**样例输出：**

OVER

**完整代码：**

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);long long a,b,limit;cin>>a>>b>>limit;__int128 prod=(__int128
```

**测试设计：**额外测试：构造最小规模、重复值、边界值各一组，和样例一起运行。

---

### V07-CEX05 空容器访问保护

- 归属卷：第 7 卷
- 覆盖模块：stack、RE 防御
- 考场用途：top/pop 前先判空。
- 参考题型来源：参考来源：括号匹配调试题型。

**题目描述：**判断只含括号的字符串是否合法。

**输入格式：**输入字符串。

**输出格式：**输出 YES/NO。

**样例输入：**

([]())

**样例输出：**

YES

**完整代码：**

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);string s;cin>>s;vector<char>st;for(char c:s){if(c=='('||c=='
```

**测试设计：**额外测试：构造最小规模、重复值、边界值各一组，和样例一起运行。

---